

Boilerplate Structure

1. Put `LICENSE` file for your license
2. At `README.md` file for all command & explanation for everything, imagine some human don't know anything and want to use your project.
3. All source code put it in `src/`

4. Add `requirements.txt` files contains all used libraries and their versions like

```
fastapi==0.141.1
uvicorn[standard]==0.52.4
```

and you can import them via `pip install -r requirements.txt`

5. Add `.env` file UPPER_CASE keys for all your configurations & secret keys, copy `.env.example` from `.env` without adding any secret data.

```
APP_NAME="Mini-RAG"
APP_VERSION="0.1"
FILE_ALLOWED_TYPES=["text/plain", "application/pdf"]
FILE_MAX_SIZE=10
```

6. Add `.gitignore` file for all files where you want to untracked by git and `.gitkeep` for uploading empty folders

7. Add folder `helpers/` containing `.env` handler like

```
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    APP_NAME: str
    FILE_ALLOWED_TYPES: list
    FILE_MAX_SIZE: int
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8")

def get_settings():
    return Settings()
```

8. Inside any folder create `__init__.py` file to collapsed importing chain like

```
from .enums.ResponseEnums import ResponseSignal
```

9. Add `assets/` contains data files like `API.yaml` & images & ..., can add `.gitignore` & `.gitkeep` files for improve tracking

10. Divide you Arch into MVC (Models Views Controllers), so create 3 dirs `Models/` everything related with data, `Views/` related with UI/UX, and `Controllers` related with logic and adding parent base inheriting class containing shared data and functions needed in any child controller

11. For all const data, define them @ Enum like

```
from enum import Enum
class ResponseSignal(Enum):
    FILE_VALIDATED_SUCCESS: str = "file_validate_successfully"
```

Code

- Make `main.py` file with minimal code, just define `app` and its routes like

```
from fastapi import FastAPI
from routes import base
app = FastAPI()
app.include_router(base.base_router)
```

- Inside `routes/` define `files.py` for each endpoint and add `schemes/` inside `routes/` to define transferred JSON objects like

```
.
├── base.py
├── data.py
├── __init__.py
└── schemes
    ├── data.py
    └── __init__.py
```

- Inside each `endpiont.py`

1. Define route with tag (for docs)

```
data_router = APIRouter(
    prefix='/api/v1/data',
    tags=['api_v1', 'data'],
)
```

2. Add logs logic by logging module

```
import logging
logger = logging.getLogger('uvicorn.error')
logger.error('Error while uploading file: ...')
```

3. If operation take data from project, define params with `Depends` fn in `fastapi` and make any endpoint `async` if needed like

```
@data_router.post('/upload/{project_id}')
async def upload_data(project_id: str, file: UploadFile,
                      app_settings: Settings = Depends(get_settings)):
    pass
```

4. make any operaiton return obj like `JSONResponse` to define status like

```
JSONResponse(
    status_code = status.HTTP_400_BAD_REQUEST,
    content = {
        'signal': ResponseSignal.FILE_UPLOAD_FAILED.value,
    }
)
```

1. FastAPI was built on Starlette so initially if loaded file size > 1MB stored on hard disk @ `/tmp/`, so to save RAM storage load chunks and this store them @ your place like

```
try:
    async with aiofiles.open(file_path, 'wb') as f:
        while chunk := await file.read(app_settings.FILE_DEFAULT_CHUNK_SIZE):
            await f.write(chunk)
except Exception as e:
    logger.error(f'Error while uploading file: {e}')
    return JSONResponse(
        status_code = status.HTTP_400_BAD_REQUEST,
        content = {
            'signal': ResponseSignal.FILE_UPLOAD_FAILED.value,
        }
    )
```